ICT Academy of Kerala · Python Programming

QuickMart Billing System

1-hour case study · Trainer solution

Working code, real output and a facilitation guide

The scenario

QuickMart is a campus store. The owner wants a billing program built from small, reusable **functions** so that prices, discounts and tax can change without rewriting the program. **You will build it.**
File name: `quickmart_billing.py` **Time:** 60 minutes

Skills you will use

- Defining functions with parameters, default values and return values
- Functions that process a list of dictionaries
- Passing a function to `sorted` with `key=`
- Formatted output and testing with `assert`

STARTER DATA (COPY EXACTLY)

```
# quickmart_billing.py - QuickMart Billing System
cart = [
    {"name": "Notebook", "category": "Stationery", "price": 40,  "qty": 5},
    {"name": "Pen",      "category": "Stationery", "price": 10,  "qty": 11},
    {"name": "Biscuits", "category": "Snacks",     "price": 25,  "qty": 4},
    {"name": "Juice",    "category": "Drinks",     "price": 45,  "qty": 3},
    {"name": "Chips",    "category": "Snacks",     "price": 20,  "qty": 6},
]
```

Functions you will write

#	Function	What it does
1	line_total(item)	returns price × qty of one item
2	subtotal(cart)	returns the sum of all line totals (use `line_total`)
3	apply_discount(amount, percent=0)	returns the amount after reducing it by percent; default 0
4	gst(amount, rate=18)	returns the GST on the amount; default rate 18
5	category_totals(cart)	returns a dictionary of category to total value
6	top_items(cart, n=2)	returns the names of the n items with the highest line total (use `sorted` with `key=line_total`)
7	print_bill(cart, discount_percent=0)	prints the bill table and the summary, using the functions above
8	main()	calls everything and prints each task heading; then ends with a few `assert` tests

Tasks and expected output

Task 1 · Line total

Print the line total of the first item in the cart.

EXPECTED OUTPUT

```
Task 1: Line total
200
```

Task 2 · Subtotal

Print the subtotal of the cart.

EXPECTED OUTPUT

```
Task 2: Subtotal
665
```

Task 3 · Discount

Print `apply_discount(500)` and `apply_discount(500, 10)`.

EXPECTED OUTPUT

```
Task 3: Discount
500.0
450.0
```

Task 4 · GST

Print `gst(100)` and `gst(100, 5)`.

EXPECTED OUTPUT

```
Task 4: GST
18.0
5.0
```

Task 5 · Category totals

Print the category totals, categories in alphabetical order, the name padded to 12 characters.

EXPECTED OUTPUT

```
Task 5: Category totals
Drinks      135
Snacks      220
Stationery  310
```

Task 6 · Top items

Print `top_items(cart)`, then `top_items(cart, 3)`.

EXPECTED OUTPUT

```
Task 6: Top items
['Notebook', 'Juice']
['Notebook', 'Juice', 'Chips']
```

Task 7 · Bill

Call `print_bill(cart, 10)`. Columns: item 10 wide (left), price 6, qty 5, total 8 (right aligned); the summary amounts are right aligned in 8 characters with 2 decimals. The bill has a 10% discount and 18% GST on the discounted amount.

EXPECTED OUTPUT

```
Task 7: Bill
Item       Price  Qty   Total
----------------------------
Notebook      40    5     200
Pen           10   11     110
Biscuits      25    4     100
Juice         45    3     135
Chips         20    6     120
----------------------------
Subtotal                665.00
Discount 10%             66.50
GST 18%                 107.73
Grand total             706.23
```

Task 8 · Tests

Write four `assert` tests (for example line_total of one item, apply_discount(200, 50), gst(200), and the top item) and print the message below if they all pass.

EXPECTED OUTPUT

```
Task 8: Tests
All tests passed
```

Print the heading lines (for example `Task 1: Line total`) exactly as shown, with a blank line between tasks. Write and run each function before moving to the next.

Complete solution

```
# quickmart_billing.py - QuickMart Billing System
cart = [
    {"name": "Notebook", "category": "Stationery", "price": 40,  "qty": 5},
    {"name": "Pen",      "category": "Stationery", "price": 10,  "qty": 11},
    {"name": "Biscuits", "category": "Snacks",     "price": 25,  "qty": 4},
    {"name": "Juice",    "category": "Drinks",     "price": 45,  "qty": 3},
    {"name": "Chips",    "category": "Snacks",     "price": 20,  "qty": 6},
]


def line_total(item):
    return item["price"] * item["qty"]


def subtotal(cart):
    return sum(line_total(item) for item in cart)


def apply_discount(amount, percent=0):
    return amount - amount * percent / 100


def gst(amount, rate=18):
    return amount * rate / 100


def category_totals(cart):
    totals = {}
    for item in cart:
        totals[item["category"]] = totals.get(item["category"], 0) + line_total(item)
    return totals


def top_items(cart, n=2):
    ranked = sorted(cart, key=line_total, reverse=True)
    return [item["name"] for item in ranked[:n]]


def print_bill(cart, discount_percent=0):
    print(f"{'Item':<10}{'Price':>6}{'Qty':>5}{'Total':>8}")
    print("-" * 29)
    for item in cart:
        print(f"{item['name']:<10}{item['price']:>6}{item['qty']:>5}{line_total(item):>8}")
    print("-" * 29)
    sub = subtotal(cart)
    after = apply_discount(sub, discount_percent)
    tax = gst(after)
    print(f"{'Subtotal':<21}{sub:>8.2f}")
    print(f"{'Discount ' + str(discount_percent) + '%':<21}{sub - after:>8.2f}")
    print(f"{'GST 18%':<21}{tax:>8.2f}")
    print(f"{'Grand total':<21}{after + tax:>8.2f}")


def main():
    print("Task 1: Line total")
    print(line_total(cart[0]))
    print("\nTask 2: Subtotal")
    print(subtotal(cart))
    print("\nTask 3: Discount")
    print(apply_discount(500))
```

```
    print(apply_discount(500, 10))
    print("\nTask 4: GST")
    print(gst(100))
    print(gst(100, 5))
    print("\nTask 5: Category totals")
    for cat, total in sorted(category_totals(cart).items()):
        print(f"{cat:<12}{total}")
    print("\nTask 6: Top items")
    print(top_items(cart))
    print(top_items(cart, 3))
    print("\nTask 7: Bill")
    print_bill(cart, 10)
    print("\nTask 8: Tests")
    assert line_total({"name": "x", "category": "y", "price": 5, "qty": 3}) == 15
    assert apply_discount(200, 50) == 100
    assert gst(200) == 36
    assert top_items(cart, 1) == ["Notebook"]
    print("All tests passed")


main()
```

Facilitation guide

Function	Concepts checked	Common mistakes to watch for
line_total, subtotal	return, calling one function from another	printing instead of returning; `sum` of prices instead of line totals
apply_discount, gst	default parameters	placing a default before a non-default parameter; changing the wrong base amount
category_totals	dictionary counting pattern inside a function	KeyError on a new category; resetting the dictionary inside the loop
top_items	`sorted(key=function)`, slicing	calling `line_total()` instead of passing `line_total`
print_bill	composition, f-string alignment	tax on the wrong amount (GST must be on the discounted amount)
main + assert	structuring a program, testing	forgetting to call `main()`

Quick marking checklist

- Output matches exactly (spacing and alignment)
- Every function has one clear job and returns rather than prints (except print_bill)
- Default parameters used as specified
- No global variables changed inside functions
- Code is readable: meaningful names, docstrings or comments

Trainer tip: Demo Functions 1 and 2 live (8 minutes). Pause after Function 4 and ask what happens if GST changes to 5%: only the default changes.